

National University of Computer and Emerging Sciences, Lahore Campus



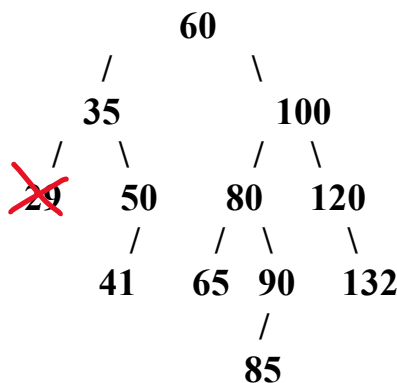
Course:	Data Structures	Course Code:	CS2001
Program:	BS (CS, SE, DS)	Semester:	Fall 2021
Duration:	60 Minutes	Total Marks:	25
Paper Date:	03-Dec-2021	Page(s):	6
Sections:	ALL	Section:	
Exam:	Midterm Exam 2	Roll No:	

Instruction/Notes: Answer in the space provided. You cannot ask for rough sheets since they are already attached. Rough sheets **will not be graded or marked**. In case of any confusion or ambiguity, make a reasonable assumption.

Question 1:

(Marks:10+5)

- a) Redraw the following AVL tree after deletion of key 29 and. You must show all working including the names of disbalanced cases, nodes, and the rotations performed.



Rotate Right z along y	Rotate Left z along x	Disbalance at root 60
Rotate Right z along y y takes ownership of z's right child	Rotate Left z along x x takes ownership of z's left child	

- b) For each of the scenarios given below, suggest the most appropriate data structure chosen from the list. (Arrays, doubly linked-list, Queue, Stack, tree)

1. to record the sequence of all the pages browsed in one session.	doubly linked-list
2. to store information about the directories and files in a system	tree
3. to implement printer spooler so that jobs can be printed in the order of their arrival.	Queue
4. to implement “back” functionality in the internet browser.	Stack
5. to store an image in the form of a bit map	Arrays

Question 2:

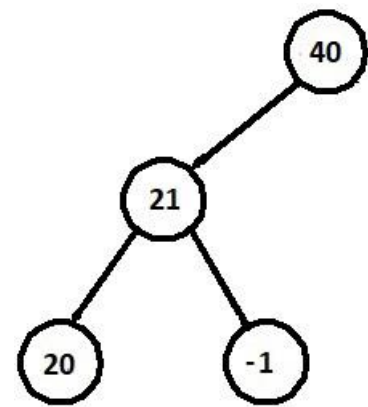
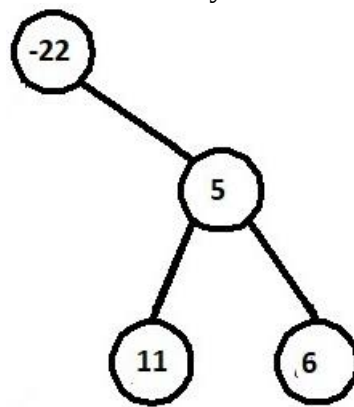
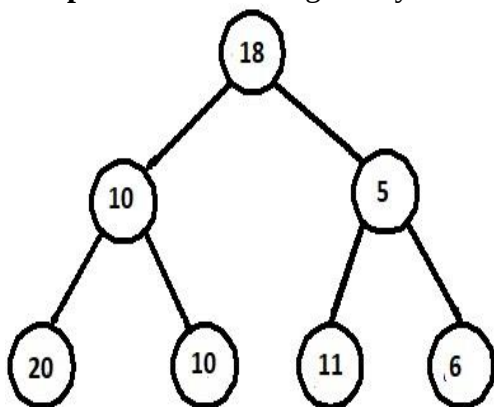
(Marks: 10)

Write a recursive C/C++ function **isDifferenceBinaryTree** which is passed the root of a binary tree as a parameter. It then checks whether a given binary tree is a **difference binary tree** or not. We define difference binary tree as a binary tree in which the difference between the sum of all keys of left subtree of a **non-leaf** node and sum of all keys of right subtree of that non-leaf node equals the key of that node. If every non-leaf node in a binary tree has that property, then the binary tree will be a difference binary tree. The sum of keys of an empty subtree will be equal to zero. If the binary tree is a difference binary tree, then return true else return false. If you want to use any helper function, then you must give its implementation as well.

Assume that Tree Node is implemented as follows:

```
class TNode
{
    int key;
    TNode* lChild;
    TNode* rChild;
};
```

Examples: The following binary trees are difference binary trees.



```
template < typename T >
bool BST<T>:: isDifferecne() {
    int sum = 0;
    return isDifferecne(root, sum);
}

template < typename T >
bool BST<T>:: isDifferecne(Node* curr, int & sum)
{
    if (curr == nullptr) {
        sum = 0;
        return true;
    }
    else
    {
        int lsum = 0, rsum = 0;
        bool l = 0, r = 0;
        l = isDifferecne(curr->left, lsum);
        r = isDifferecne(curr->right, rsum);
        sum = curr->data + lsum + rsum;

        cout << curr->data << " ";
        cout << rsum - lsum << " ";
        cout << sum << endl ;

        if (l && r && (curr->data == (rsum - lsum)))
            return true;
        else
            return false;
    }
}
```

Roll Number: _____

Section: _____

Roll Number: _____

Section: _____

Rough Sheet

Rough Sheet

Roll Number: _____

Section: _____

Rough Sheet